

Task 2

Task 2

Tasks

- Solve the gridworld environment `AsteroidStatic` in file `asteroid.py` with the smallest number of `_calc_next_state` calls (or variations thereof).
- Solve the gridworld environment `AsteroidStatic` in file `asteroid.py` with the smallest number of time steps per episode (a.k.a. how fast can you reach the goal).
- Your report includes three numbers, based on the average of 10 different environments:
 - number of `_calc_next_state` calls after which the environment has been solved for the first time
 - minimum number of episode time steps to solve the environment and (not graded) the associated number of `_calc_next_state` calls until you achieved this goal

Deadline: 19.07.2026 23:59

Hints:

- You are allowed to use any public function of the environment
- You are allowed to look at the existing code to understand how the environment works
- Have a look at [Optuna](#) for optimizing your hyperparameters automatically (e.g. ϵ , α , ...)
- Do not use the same environments (e.g. same seed) for hyperparameter optimization / final plots

Task 2

Notes on task 2:

- All solutions must be implemented by yourself (not in a group) in Python 3
- Do not modify any of the provided Python files
- Use the default parameterization of the environment
- Do not analyze the environment algorithmically (or ask me first if you have a specific idea in mind)
- Your Moodle submission includes the following:
 - your code (Jupyter notebook or Python file)
 - a readme.txt explaining how to run the code
 - a pdf/doc(x) file containing at most five pages (including images/plots) presenting the approaches you tried out, corresponding results and your final best solution. You are encouraged to present approaches/ideas that did not work well, too. Also include a description of your newly created environment and a plot that it can be solved.